

Name: Apurba Koirala

22BCE3799

Deep Learning Lab Assessment 2

Guided by: Dr. C Sharmila

1. Implement a Convolutional Neural Network (CNN) for face recognition using a labeled face image dataset such as LFW, CelebA, or a custom dataset. The model should classify images to accurately identify different individuals.

```
from sklearn.datasets import fetch_lfw_people
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import numpy as np

lfw_people = fetch_lfw_people(min_faces_per_person=50, resize=1.0, color =
False)

lfw = fetch_lfw_people(min_faces_per_person=50, resize=1.0, color=False)
images, labels = lfw_people.images, lfw_people.target
target_names = lfw_people.target_names

X = images.astype("float32")
if X.max() > 2.0:
    X /= 255.0
X = np.expand_dims(X, -1)
y = labels

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

print(f"\nDataset split:")
print(f"X_train shape: {X_train.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")
```

Dataset split:

X_train shape: (1248, 125, 94, 1)

y_train shape: (1248,)

X_test shape: (312, 125, 94, 1)

y_test shape: (312,)

```
from sklearn.preprocessing import LabelBinarizer
lb = LabelBinarizer()
y_train_ohe = lb.fit_transform(y_train)
y_test_ohe = lb.transform(y_test)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Create a Sequential model
model = Sequential()

# Add the first Convolutional layer
model.add(Conv2D(filters=32, kernel_size=(3, 3), activation='relu',
input_shape=(h, w, 1)))

# Add the first MaxPooling layer
model.add(MaxPooling2D(pool_size=(2, 2)))

# Add the second Convolutional layer
model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))

# Add the second MaxPooling layer
model.add(MaxPooling2D(pool_size=(2, 2)))

# Add a Flatten layer
model.add(Flatten())

# Add a Dense layer
model.add(Dense(units=128, activation='relu'))

# Add the output Dense layer
model.add(Dense(units=n_classes, activation='softmax'))

# Print the model summary
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 48, 35, 32)	320
max_pooling2d (MaxPooling2D)	(None, 24, 17, 32)	0
conv2d_1 (Conv2D)	(None, 22, 15, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 11, 7, 64)	0
flatten (Flatten)	(None, 4928)	0
dense (Dense)	(None, 128)	630,912
dense_1 (Dense)	(None, 7)	903

Total params: 650,631 (2.48 MB)
Trainable params: 650,631 (2.48 MB)
Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

Model compiled successfully with:
Optimizer: adam
Loss: SparseCategoricalCrossentropy
Metrics: ['loss', 'compile_metrics']

```
# Define training parameters
epochs = 10
batch_size = 32

# Train the model
history = model.fit(X_train, y_train_one, epochs=epochs,
                    batch_size=batch_size, validation_split=0.1, verbose=1)

# Display the history object
print("Training history:")
print(history.history.keys())
```

```

Epoch 1/10
36/36 ————— 9s 156ms/step - accuracy: 0.2566 - loss: 2.3462 - val_accuracy: 0.3520 - val_loss: 2.1868
Epoch 2/10
36/36 ————— 1s 15ms/step - accuracy: 0.3334 - loss: 2.1940 - val_accuracy: 0.3520 - val_loss: 2.1236
Epoch 3/10
36/36 ————— 1s 15ms/step - accuracy: 0.3401 - loss: 2.1345 - val_accuracy: 0.3920 - val_loss: 1.9108
Epoch 4/10
36/36 ————— 1s 16ms/step - accuracy: 0.3548 - loss: 1.9683 - val_accuracy: 0.4480 - val_loss: 1.6081
Epoch 5/10
36/36 ————— 1s 17ms/step - accuracy: 0.4693 - loss: 1.5965 - val_accuracy: 0.6240 - val_loss: 1.1278
Epoch 6/10
36/36 ————— 1s 24ms/step - accuracy: 0.5433 - loss: 1.3026 - val_accuracy: 0.6880 - val_loss: 0.9037
Epoch 7/10
36/36 ————— 1s 16ms/step - accuracy: 0.6922 - loss: 0.9620 - val_accuracy: 0.7600 - val_loss: 0.7615
Epoch 8/10
36/36 ————— 1s 16ms/step - accuracy: 0.7390 - loss: 0.7704 - val_accuracy: 0.8000 - val_loss: 0.7012
Epoch 9/10
36/36 ————— 1s 16ms/step - accuracy: 0.7861 - loss: 0.6381 - val_accuracy: 0.8160 - val_loss: 0.6853
Epoch 10/10
36/36 ————— 1s 15ms/step - accuracy: 0.8516 - loss: 0.4699 - val_accuracy: 0.8320 - val_loss: 0.6783
Training history:
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])

```

```

loss, acc = model.evaluate(X_test, y_test_one, verbose=0)
print(f"Test accuracy: {acc:.4f}")

```

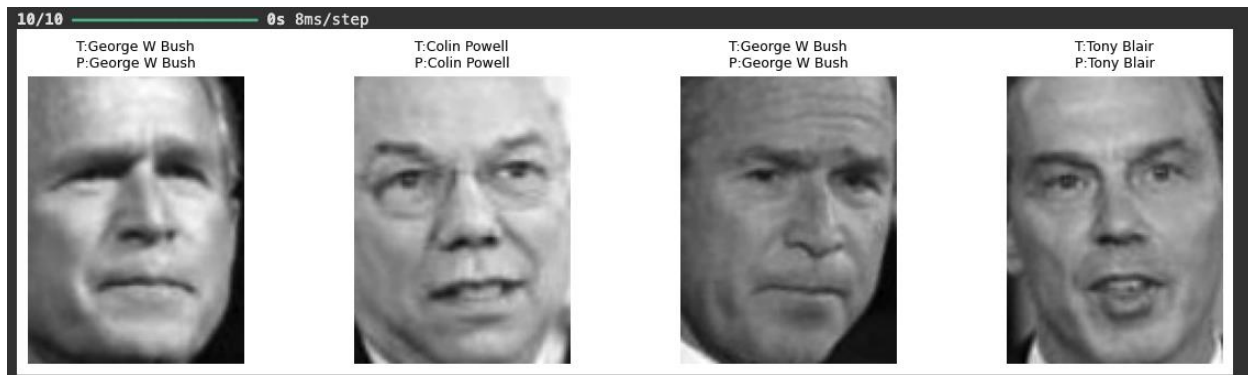
Test accuracy: 0.8141

```

import matplotlib.pyplot as plt
y_pred = np.argmax(model.predict(X_test), axis=1)
def show_predictions(Xs, ys_true, ys_pred, names, n=10):
    plt.figure(figsize=(15,6))
    for i in range(n):
        plt.subplot(2, 5, i+1)
        plt.imshow(Xs[i].squeeze(), cmap="gray")
        plt.title(f"T:{names[ys_true[i]]}\nP:{names[ys_pred[i]]}",
fontsize=9)
        plt.axis("off")
    plt.tight_layout()

show_predictions(X_test, y_test, y_pred, target_names, n=4)

```



2. Download the Churn Modeling dataset.

a. Apply deep learning techniques to train and test the data using a 70:30 split.

- Design the model with hyperparameter tuning for hidden layers, output layer, dropout rates, and activation functions.
- Plot the comparative graph of the performance (at least three best performance of hyper parameter tuning)

b. Read an .jpg or .png image and get the following output

- Print original image
- Print the feature map after convolution layer (tensor and image)
- Print the feature map after the pooling layer
- Provide an analysis of the feature extraction process based on the results.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
```

```
import cv2
```

```
df = pd.read_csv("Churn_Modelling.csv")
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Features and target
X = df.drop(columns=["RowNumber", "CustomerId", "Surname", "Exited"])
y = df["Exited"]

# Encode categorical features
le_geo = LabelEncoder()
X["Geography"] = le_geo.fit_transform(X["Geography"])

le_gen = LabelEncoder()
X["Gender"] = le_gen.fit_transform(X["Gender"])

# Train test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Train shape:", X_train.shape, "Test shape:", X_test.shape)
```

```
Train shape: (7000, 10) Test shape: (3000, 10)
```

```
def build_model(units1=64, units2=32, dropout=0.3, activation='relu'):
    model = models.Sequential([
        layers.Dense(units1, activation=activation,
input_shape=(X_train.shape[1],)),
        layers.Dropout(dropout),
        layers.Dense(units2, activation=activation),
        layers.Dropout(dropout),
        layers.Dense(1, activation="sigmoid")
    ])
    model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])
    return model
```

```
# Different hyperparameter setups
configs = [
    {"units1":64, "units2":32, "dropout":0.3, "activation":"relu"},
    {"units1":128,"units2":64, "dropout":0.4, "activation":"tanh"},
    {"units1":256,"units2":128,"dropout":0.5, "activation":"relu"}
]

histories = []
results = []

for i, cfg in enumerate(configs):
    print(f"\nTraining Model {i+1} with config: {cfg}")
    model = build_model(**cfg)
    history = model.fit(X_train, y_train, validation_data=(X_test,y_test),
epochs=20, batch_size=32, verbose=0)
    loss, acc = model.evaluate(X_test, y_test, verbose=0)
    results.append({"config": cfg, "accuracy": acc})
    histories.append(history)
```

```
Training Model 1 with config: {'units1': 64, 'units2': 32, 'dropout': 0.3, 'activation': 'relu'}
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

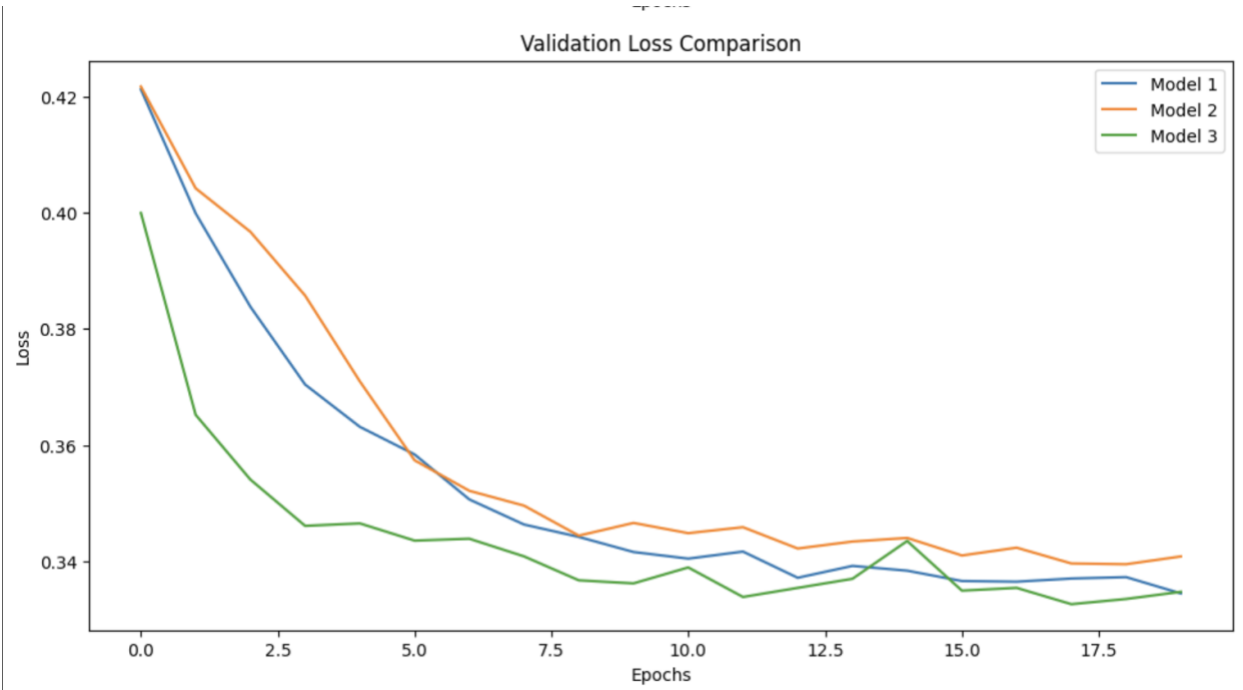
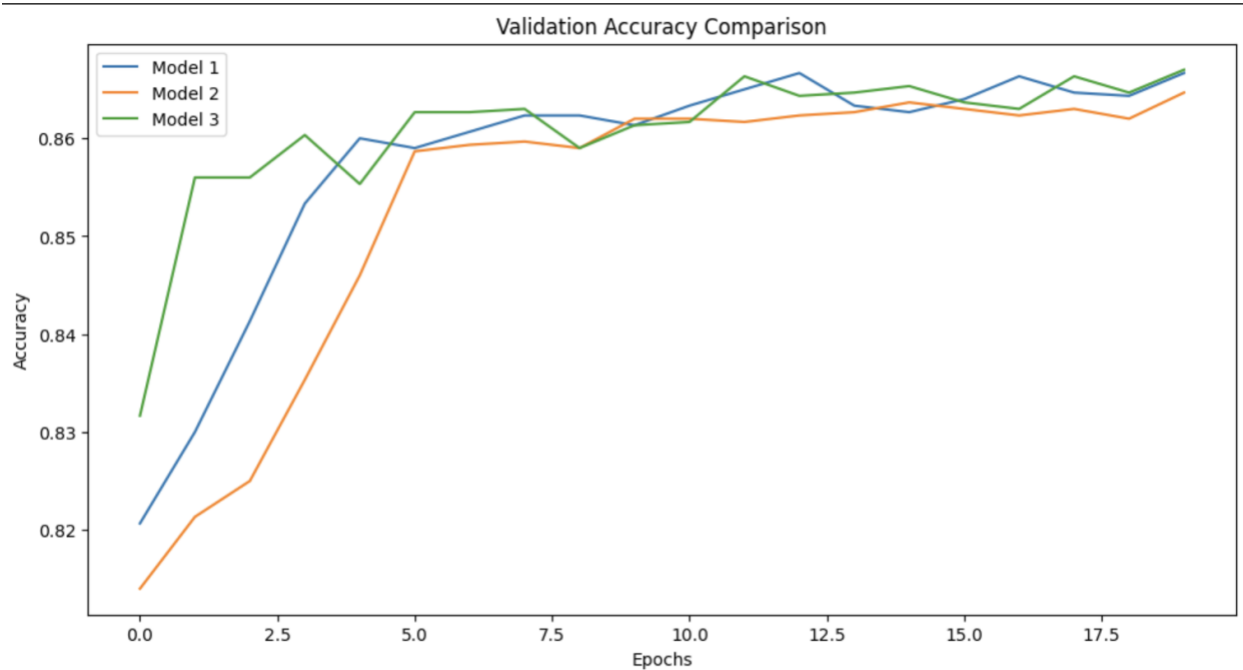
Training Model 2 with config: {'units1': 128, 'units2': 64, 'dropout': 0.4, 'activation': 'tanh'}

Training Model 3 with config: {'units1': 256, 'units2': 128, 'dropout': 0.5, 'activation': 'relu'}
```

```
plt.figure(figsize=(12,6))
for i, history in enumerate(histories):
    plt.plot(history.history['val_accuracy'], label=f'Model {i+1}')
plt.title("Validation Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Plot loss
plt.figure(figsize=(12,6))
for i, history in enumerate(histories):
    plt.plot(history.history['val_loss'], label=f'Model {i+1}')
plt.title("Validation Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

```
print("\nFinal Test Accuracies:")
for r in results:
    print(r)
```



```
file_path = "jorse.jpeg"
img = cv2.imread(file_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
img = cv2.resize(img, (128,128))
img_tensor = np.expand_dims(img / 255.0, axis=0)
```

```
inputs = layers.Input(shape=(128,128,3))
x = layers.Conv2D(16, (3,3), activation='relu')(inputs)
x_pool = layers.MaxPooling2D((2,2))(x)

model = models.Model(inputs=inputs, outputs=x_pool)
```

```
conv_model = models.Model(inputs=inputs, outputs=x)
pool_model = models.Model(inputs=inputs, outputs=x_pool)
```

```
conv_features = conv_model.predict(img_tensor)
pool_features = pool_model.predict(img_tensor)

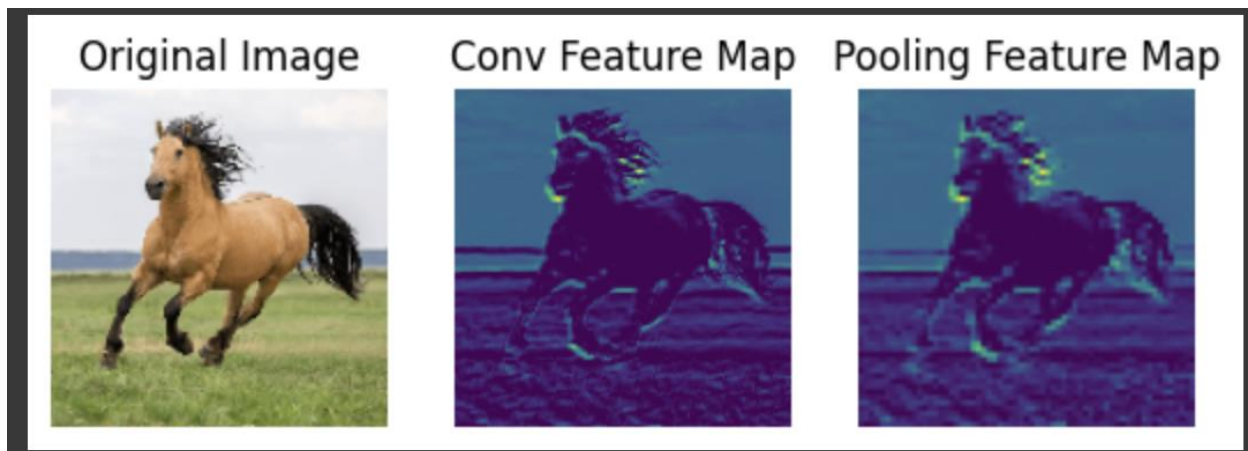
print("Conv feature map shape:", conv_features.shape)
print("Pooling feature map shape:", pool_features.shape)
```

```
# Original image
plt.subplot(1,3,1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

# Conv feature map (first filter)
plt.subplot(1,3,2)
plt.imshow(conv_features[0,:,:,:0], cmap="viridis")
plt.title("Conv Feature Map")
plt.axis("off")

# Pooling feature map (first filter)
plt.subplot(1,3,3)
plt.imshow(pool_features[0,:,:,:0], cmap="viridis")
plt.title("Pooling Feature Map")
plt.axis("off")

plt.show()
```



3. Implement a Convolutional Neural Network (CNN) using Keras to classify images from the CIFAR-10 dataset. Load the dataset, normalize the images, and apply one-hot encoding to the labels. Build a CNN with at least three convolutional layers, pooling layers, a flatten layer, and a dense output layer with softmax activation. Compile the model using the Adam optimizer and categorical crossentropy loss, then train it for 20 epochs. Plot accuracy and loss curves, evaluate the model on the test set, and print the final test accuracy.

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
```

```
# Load the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize pixel values to be between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# One-hot encode the labels
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)
```

```

model = models.Sequential([
    layers.Conv2D(32, (3,3), activation="relu", padding="same",
input_shape=x_train.shape[1:]),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation="relu", padding="same"),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(128, (3,3), activation="relu", padding="same"),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.4),
    layers.Dense(10, activation="softmax")
])

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
h = model.fit(x_train, y_train, epochs=20, validation_data=(x_test,
y_test))

# Plot accuracy and loss curves
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(h.history['accuracy'], label='accuracy')
plt.plot(h.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(h.history['loss'], label='loss')
plt.plot(h.history['val_loss'], label='val_loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

```

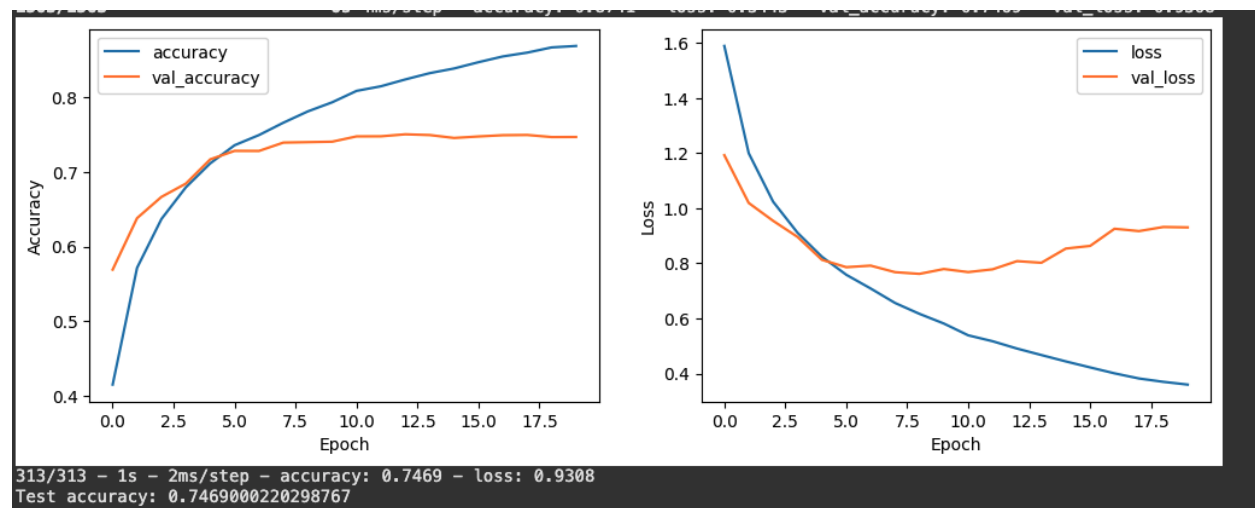
```
plt.show()
```

```
# Evaluate the model
```

```
loss, acc = model.evaluate(x_test, y_test, verbose=2)
```

```
print(f"Test accuracy: {acc}")
```

```
Epoch 1/20  
1563/1563 — 15s 6ms/step - accuracy: 0.3185 - loss: 1.8281 - val_accuracy: 0.5691 - val_loss: 1.1930  
Epoch 2/20  
1563/1563 — 6s 4ms/step - accuracy: 0.5535 - loss: 1.2422 - val_accuracy: 0.6381 - val_loss: 1.0195  
Epoch 3/20  
1563/1563 — 10s 4ms/step - accuracy: 0.6302 - loss: 1.0387 - val_accuracy: 0.6666 - val_loss: 0.9546  
Epoch 4/20  
1563/1563 — 6s 4ms/step - accuracy: 0.6750 - loss: 0.9227 - val_accuracy: 0.6844 - val_loss: 0.8963  
Epoch 5/20  
1563/1563 — 10s 4ms/step - accuracy: 0.7124 - loss: 0.8237 - val_accuracy: 0.7169 - val_loss: 0.8131  
Epoch 6/20  
1563/1563 — 10s 4ms/step - accuracy: 0.7421 - loss: 0.7437 - val_accuracy: 0.7283 - val_loss: 0.7862  
Epoch 7/20  
1563/1563 — 11s 4ms/step - accuracy: 0.7513 - loss: 0.6946 - val_accuracy: 0.7282 - val_loss: 0.7918  
Epoch 8/20  
1563/1563 — 6s 4ms/step - accuracy: 0.7707 - loss: 0.6442 - val_accuracy: 0.7394 - val_loss: 0.7681  
Epoch 9/20  
1563/1563 — 10s 4ms/step - accuracy: 0.7834 - loss: 0.6015 - val_accuracy: 0.7400 - val_loss: 0.7619  
Epoch 10/20  
1563/1563 — 10s 4ms/step - accuracy: 0.8019 - loss: 0.5593 - val_accuracy: 0.7406 - val_loss: 0.7795  
Epoch 10/20  
1563/1563 — 10s 4ms/step - accuracy: 0.8019 - loss: 0.5593 - val_accuracy: 0.7406 - val_loss: 0.7795  
Epoch 11/20  
1563/1563 — 10s 4ms/step - accuracy: 0.8132 - loss: 0.5243 - val_accuracy: 0.7477 - val_loss: 0.7684  
Epoch 12/20  
1563/1563 — 10s 4ms/step - accuracy: 0.8190 - loss: 0.5029 - val_accuracy: 0.7478 - val_loss: 0.7784  
Epoch 13/20  
1563/1563 — 11s 4ms/step - accuracy: 0.8266 - loss: 0.4792 - val_accuracy: 0.7505 - val_loss: 0.8081  
Epoch 14/20  
1563/1563 — 11s 4ms/step - accuracy: 0.8394 - loss: 0.4452 - val_accuracy: 0.7495 - val_loss: 0.8023  
Epoch 15/20  
1563/1563 — 10s 4ms/step - accuracy: 0.8438 - loss: 0.4268 - val_accuracy: 0.7456 - val_loss: 0.8538  
Epoch 16/20  
1563/1563 — 6s 4ms/step - accuracy: 0.8504 - loss: 0.4113 - val_accuracy: 0.7476 - val_loss: 0.8635  
Epoch 17/20  
1563/1563 — 6s 4ms/step - accuracy: 0.8598 - loss: 0.3885 - val_accuracy: 0.7493 - val_loss: 0.9259  
Epoch 18/20  
1563/1563 — 6s 4ms/step - accuracy: 0.8674 - loss: 0.3635 - val_accuracy: 0.7496 - val_loss: 0.9173  
Epoch 19/20  
1563/1563 — 10s 4ms/step - accuracy: 0.8724 - loss: 0.3506 - val_accuracy: 0.7468 - val_loss: 0.9322  
Epoch 20/20  
1563/1563 — 6s 4ms/step - accuracy: 0.8741 - loss: 0.3443 - val_accuracy: 0.7469 - val_loss: 0.9308
```



The model has overfitted, early stopping to be applied for better results.

4. Download any medical imaging dataset and compare the performance of AlexNet, ResNet, and InceptionNet architectures on the same dataset. Record the training time, training accuracy, validation accuracy, and test accuracy for each model. Plot and compare the graphs of Training vs. Validation Accuracy and Training vs. Validation Loss for all three models. Analyze and report which architecture achieved the best performance based on the results.

```
import time
import numpy as np
import tensorflow as tf
from tensorflow.keras import Input, Model, Sequential
from tensorflow.keras.layers import (
    Conv2D, MaxPooling2D, Flatten, Dense, Dropout, Activation,
    BatchNormalization, AveragePooling2D, concatenate,
    GlobalAveragePooling2D
)
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
```

```
fashion_mnist = tf.keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) =
fashion_mnist.load_data()
all_images = np.concatenate([train_images, test_images],
axis=0).astype('float32') / 255.0
all_labels = np.concatenate([train_labels, test_labels], axis=0)
all_images = np.expand_dims(all_images, axis=-1) # shape (N,28,28,1)

# Stratified splits: 70/15/15
X_train, X_temp, y_train, y_temp = train_test_split(
    all_images, all_labels, test_size=0.30, random_state=42,
    stratify=all_labels
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
```

```
)

print("Shapes:", X_train.shape, X_val.shape, X_test.shape)
```

```
input_shape = (28, 28, 1)
num_classes = 10
```

Three functions for the three transfer learning models

```
def build_simplified_alexnet_adjusted(input_shape, num_classes):
    model = Sequential([
        Conv2D(32, (3, 3), strides=(1, 1), activation='relu',
input_shape=input_shape, padding='same'),
        MaxPooling2D((2, 2)),
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Flatten(),
        Dense(512, activation='relu'),
        Dropout(0.5),
        Dense(512, activation='relu'),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])
    return model
```

```
def build_simplified_resnet(input_shape, num_classes):
    from tensorflow.keras.layers import Reshape # local import if needed
    def identity_block(x, filters, kernel_size):
        x_skip = x
        x = Conv2D(filters, kernel_size, padding='same')(x)
        x = BatchNormalization(axis=3)(x)
        x = Activation('relu')(x)
        x = Conv2D(filters, kernel_size, padding='same')(x)
        x = BatchNormalization(axis=3)(x)
        x = concatenate([x, x_skip])
        x = Activation('relu')(x)
        return x

    def convolutional_block(x, filters, kernel_size, strides=(2, 2)):

```

```

        x_skip = x
        x = Conv2D(filters, kernel_size, strides=strides,
padding='same')(x)
        x = BatchNormalization(axis=3)(x)
        x = Activation('relu')(x)
        x = Conv2D(filters, kernel_size, padding='same')(x)
        x = BatchNormalization(axis=3)(x)
        x_skip = Conv2D(filters, (1, 1), strides=strides,
padding='same')(x_skip)
        x_skip = BatchNormalization(axis=3)(x_skip)
        x = concatenate([x, x_skip])
        x = Activation('relu')(x)
        return x

input_tensor = Input(shape=input_shape)
x = Conv2D(32, (3, 3), strides=(1, 1), padding='same')(input_tensor)
x = BatchNormalization(axis=3)(x)
x = Activation('relu')(x)
x = MaxPooling2D((3, 3), strides=(2, 2), padding='same')(x)

x = convolutional_block(x, 32, (3, 3), strides=(1,1)) # avoid too
aggressive downsample
x = identity_block(x, 32, (3, 3))

x = convolutional_block(x, 64, (3, 3), strides=(1,1))
x = identity_block(x, 64, (3, 3))

# FIX: use adaptive global pooling instead of fixed 7x7
x = GlobalAveragePooling2D(name='global_avg_pool')(x) # -> (batch,
channels)
x = Dense(num_classes, activation='softmax', name='fc10')(x)

model = Model(inputs=input_tensor, outputs=x,
name='resnet_simplified')
return model

```

```

def build_simplified_inception(input_shape, num_classes):
    def inception_module(x, filters_1x1, filters_3x3_reduce, filters_3x3,
filters_5x5_reduce, filters_5x5, filters_pool_proj):
        conv_1x1 = Conv2D(filters_1x1, (1, 1), padding='same',
activation='relu')(x)

```

```

        conv_3x3 = Conv2D(filters_3x3_reduce, (1, 1), padding='same',
activation='relu')(x)
        conv_3x3 = Conv2D(filters_3x3, (3, 3), padding='same',
activation='relu')(conv_3x3)

        conv_5x5 = Conv2D(filters_5x5_reduce, (1, 1), padding='same',
activation='relu')(x)
        conv_5x5 = Conv2D(filters_5x5, (5, 5), padding='same',
activation='relu')(conv_5x5)

        pool_proj = MaxPooling2D((3, 3), strides=(1, 1),
padding='same')(x)
        pool_proj = Conv2D(filters_pool_proj, (1, 1), padding='same',
activation='relu')(pool_proj)

        output = concatenate([conv_1x1, conv_3x3, conv_5x5, pool_proj],
axis=3)
        return output

input_tensor = Input(shape=input_shape)

x = Conv2D(32, (3, 3), strides=(1, 1), padding='same',
activation='relu')(input_tensor)
x = MaxPooling2D((2, 2), strides=(2, 2), padding='same')(x)

x = Conv2D(64, (1, 1), padding='same', activation='relu')(x)
x = Conv2D(96, (3, 3), padding='same', activation='relu')(x)
x = MaxPooling2D((2, 2), strides=(2, 2), padding='same')(x)

x = inception_module(x, 32, 48, 64, 8, 16, 16)
x = inception_module(x, 64, 64, 96, 16, 48, 32)
x = MaxPooling2D((2, 2), strides=(2, 2), padding='same')(x)

x = GlobalAveragePooling2D()(x)
x = Dense(num_classes, activation='softmax')(x)

model = Model(inputs=input_tensor, outputs=x,
name='inception_simplified')
return model

```

```

alexnet_model = build_simplified_alexnet_adjusted(input_shape,
num_classes)
resnet_model = build_simplified_resnet(input_shape, num_classes)
inception_model = build_simplified_inception(input_shape, num_classes)

```

```
for m in (alexnet_model, resnet_model, inception_model):
    m.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

```
print("\nAlexNet summary:")
alexnet_model.summary()
```

AlexNet summary:
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	73,856
conv2d_3 (Conv2D)	(None, 7, 7, 128)	147,584
conv2d_4 (Conv2D)	(None, 7, 7, 64)	73,792
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 64)	0
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 512)	295,424
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 512)	262,656
dropout_1 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 10)	5,130

Total params: 877,258 (3.35 MB)
Trainable params: 877,258 (3.35 MB)
Non-trainable params: 0 (0.00 B)

```
print("\nResNet summary:")
resnet_model.summary()
```


ResNet summary:
Model: "resnet_simplified"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0	-
conv2d_5 (Conv2D)	(None, 28, 28, 32)	320	input_layer_1[0]...
batch_normalization (BatchNormalizatio...	(None, 28, 28, 32)	128	conv2d_5[0][0]
activation (Activation)	(None, 28, 28, 32)	0	batch_normalizat...
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 32)	0	activation[0][0]
conv2d_6 (Conv2D)	(None, 14, 14, 32)	9,248	max_pooling2d_3[...
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 32)	128	conv2d_6[0][0]
activation_1 (Activation)	(None, 14, 14, 32)	0	batch_normalizat...
conv2d_7 (Conv2D)	(None, 14, 14, 32)	9,248	activation_1[0][...
conv2d_8 (Conv2D)	(None, 14, 14, 32)	1,056	max_pooling2d_3[...
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 32)	128	conv2d_7[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 32)	128	conv2d_8[0][0]
concatenate (Concatenate)	(None, 14, 14, 64)	0	batch_normalizat... batch_normalizat...
activation_2 (Activation)	(None, 14, 14, 64)	0	concatenate[0][0]
conv2d_9 (Conv2D)	(None, 14, 14, 32)	18,464	activation_2[0][...
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 32)	128	conv2d_9[0][0]
activation_3 (Activation)	(None, 14, 14, 32)	0	batch_normalizat...

conv2d_10 (Conv2D)	(None, 14, 14, 32)	9,248	activation_3[0][...]
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 32)	128	conv2d_10[0][0]
concatenate_1 (Concatenate)	(None, 14, 14, 96)	0	batch_normalizat... activation_2[0][...]
activation_4 (Activation)	(None, 14, 14, 96)	0	concatenate_1[0]...
conv2d_11 (Conv2D)	(None, 14, 14, 64)	55,360	activation_4[0][...]
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 64)	256	conv2d_11[0][0]
activation_5 (Activation)	(None, 14, 14, 64)	0	batch_normalizat...
conv2d_12 (Conv2D)	(None, 14, 14, 64)	36,928	activation_5[0][...]
conv2d_13 (Conv2D)	(None, 14, 14, 64)	6,208	activation_4[0][...]
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 64)	256	conv2d_12[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 64)	256	conv2d_13[0][0]
concatenate_2 (Concatenate)	(None, 14, 14, 128)	0	batch_normalizat... batch_normalizat...
activation_6 (Activation)	(None, 14, 14, 128)	0	concatenate_2[0]...
conv2d_14 (Conv2D)	(None, 14, 14, 64)	73,792	activation_6[0][...]
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 64)	256	conv2d_14[0][0]
activation_7 (Activation)	(None, 14, 14, 64)	0	batch_normalizat...
conv2d_15 (Conv2D)	(None, 14, 14, 64)	36,928	activation_7[0][...]
batch_normalizatio... (BatchNormalizatio...	(None, 14, 14, 64)	256	conv2d_15[0][0]
concatenate_3 (Concatenate)	(None, 14, 14, 192)	0	batch_normalizat... activation_6[0][...]

activation_8 (Activation)	(None, 14, 14, 192)	0	concatenate_3[0]...
global_avg_pool (GlobalAveragePool...	(None, 192)	0	activation_8[0][...
fc10 (Dense)	(None, 10)	1,930	global_avg_pool[...

Total params: 260,778 (1018.66 KB)
 Trainable params: 259,754 (1014.66 KB)
 Non-trainable params: 1,024 (4.00 KB)

```

print("\nInception summary:")
inception_model.summary()

```

Inception summary:
Model: "inception_simplified"

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 28, 28, 1)	0	-
conv2d_16 (Conv2D)	(None, 28, 28, 32)	320	input_layer_2[0]...
max_pooling2d_4 (MaxPooling2D)	(None, 14, 14, 32)	0	conv2d_16[0][0]
conv2d_17 (Conv2D)	(None, 14, 14, 64)	2,112	max_pooling2d_4[...
conv2d_18 (Conv2D)	(None, 14, 14, 96)	55,392	conv2d_17[0][0]
max_pooling2d_5 (MaxPooling2D)	(None, 7, 7, 96)	0	conv2d_18[0][0]
conv2d_20 (Conv2D)	(None, 7, 7, 48)	4,656	max_pooling2d_5[...
conv2d_22 (Conv2D)	(None, 7, 7, 8)	776	max_pooling2d_5[...
max_pooling2d_6 (MaxPooling2D)	(None, 7, 7, 96)	0	max_pooling2d_5[...
conv2d_19 (Conv2D)	(None, 7, 7, 32)	3,104	max_pooling2d_5[...
conv2d_21 (Conv2D)	(None, 7, 7, 64)	27,712	conv2d_20[0][0]
conv2d_23 (Conv2D)	(None, 7, 7, 16)	3,216	conv2d_22[0][0]
conv2d_24 (Conv2D)	(None, 7, 7, 16)	1,552	max_pooling2d_6[...
concatenate_4 (Concatenate)	(None, 7, 7, 128)	0	conv2d_19[0][0], conv2d_21[0][0], conv2d_23[0][0], conv2d_24[0][0]
conv2d_26 (Conv2D)	(None, 7, 7, 64)	8,256	concatenate_4[0]...
conv2d_28 (Conv2D)	(None, 7, 7, 16)	2,064	concatenate_4[0]...
max_pooling2d_7 (MaxPooling2D)	(None, 7, 7, 128)	0	concatenate_4[0]...
conv2d_25 (Conv2D)	(None, 7, 7, 64)	8,256	concatenate_4[0]...
conv2d_27 (Conv2D)	(None, 7, 7, 96)	55,392	conv2d_26[0][0]
conv2d_29 (Conv2D)	(None, 7, 7, 48)	19,248	conv2d_28[0][0]

conv2d_30 (Conv2D)	(None, 7, 7, 32)	4,128	max_pooling2d_7[...
concatenate_5 (Concatenate)	(None, 7, 7, 240)	0	conv2d_25[0][0], conv2d_27[0][0], conv2d_29[0][0], conv2d_30[0][0]
max_pooling2d_8 (MaxPooling2D)	(None, 4, 4, 240)	0	concatenate_5[0]...
global_average_poo... (GlobalAveragePool...	(None, 240)	0	max_pooling2d_8[...
dense_3 (Dense)	(None, 10)	2,410	global_average_p...

Total params: 198,594 (775.76 KB)
Trainable params: 198,594 (775.76 KB)
Non-trainable params: 0 (0.00 B)

```

EPOCHS = 15
BATCH_SIZE = 128

```

```

histories = {}
times = {}

```

```

print("\nTraining AlexNet...")
t0 = time.time()
histories['alexnet'] = alexnet_model.fit(X_train, y_train,
                                         epochs=EPOCHS,
                                         batch_size=BATCH_SIZE,
                                         validation_data=(X_val, y_val),
                                         verbose=1)

```

```

Training AlexNet...
Epoch 1/15
383/383 ————— 16s 25ms/step - accuracy: 0.6346 - loss: 0.9685 - val_accuracy: 0.8653 - val_loss: 0.3726
Epoch 2/15
383/383 ————— 9s 8ms/step - accuracy: 0.8614 - loss: 0.3771 - val_accuracy: 0.8904 - val_loss: 0.3087
Epoch 3/15
383/383 ————— 5s 9ms/step - accuracy: 0.8934 - loss: 0.2953 - val_accuracy: 0.8930 - val_loss: 0.2869
Epoch 4/15
383/383 ————— 5s 9ms/step - accuracy: 0.9068 - loss: 0.2540 - val_accuracy: 0.9085 - val_loss: 0.2549
Epoch 5/15
383/383 ————— 3s 8ms/step - accuracy: 0.9131 - loss: 0.2374 - val_accuracy: 0.9142 - val_loss: 0.2365
Epoch 6/15
383/383 ————— 3s 8ms/step - accuracy: 0.9209 - loss: 0.2151 - val_accuracy: 0.9172 - val_loss: 0.2284
Epoch 7/15
383/383 ————— 3s 9ms/step - accuracy: 0.9320 - loss: 0.1873 - val_accuracy: 0.9147 - val_loss: 0.2348
Epoch 8/15
383/383 ————— 5s 8ms/step - accuracy: 0.9385 - loss: 0.1687 - val_accuracy: 0.9214 - val_loss: 0.2170
Epoch 9/15
383/383 ————— 3s 8ms/step - accuracy: 0.9397 - loss: 0.1627 - val_accuracy: 0.9267 - val_loss: 0.2167
Epoch 10/15
383/383 ————— 5s 9ms/step - accuracy: 0.9465 - loss: 0.1468 - val_accuracy: 0.9244 - val_loss: 0.2249
Epoch 11/15
383/383 ————— 5s 8ms/step - accuracy: 0.9509 - loss: 0.1338 - val_accuracy: 0.9289 - val_loss: 0.2061
Epoch 12/15
383/383 ————— 6s 10ms/step - accuracy: 0.9547 - loss: 0.1208 - val_accuracy: 0.9220 - val_loss: 0.2400
Epoch 13/15
383/383 ————— 3s 8ms/step - accuracy: 0.9591 - loss: 0.1118 - val_accuracy: 0.9192 - val_loss: 0.2552
Epoch 14/15
383/383 ————— 3s 8ms/step - accuracy: 0.9589 - loss: 0.1067 - val_accuracy: 0.9280 - val_loss: 0.2586
Epoch 15/15
383/383 ————— 6s 10ms/step - accuracy: 0.9672 - loss: 0.0903 - val_accuracy: 0.9248 - val_loss: 0.2605

```

```

times['alexnet'] = time.time() - t0
print("AlexNet time:", times['alexnet'])

```

AlexNet time: 564.4991178512573

```

print("\nTraining ResNet...")
t0 = time.time()
histories['resnet'] = resnet_model.fit(X_train, y_train,
                                      epochs=EPOCHS,
                                      batch_size=BATCH_SIZE,
                                      validation_data=(X_val, y_val),
                                      verbose=1)

times['resnet'] = time.time() - t0
print("ResNet time:", times['resnet'])

```

```

Training ResNet...
Epoch 1/15
383/383 ————— 28s 39ms/step - accuracy: 0.7406 - loss: 0.7567 - val_accuracy: 0.2476 - val_loss: 4.6114
Epoch 2/15
383/383 ————— 26s 17ms/step - accuracy: 0.8876 - loss: 0.3199 - val_accuracy: 0.8634 - val_loss: 0.3995
Epoch 3/15
383/383 ————— 10s 17ms/step - accuracy: 0.8999 - loss: 0.2747 - val_accuracy: 0.8667 - val_loss: 0.3777
Epoch 4/15
383/383 ————— 7s 17ms/step - accuracy: 0.9134 - loss: 0.2414 - val_accuracy: 0.7655 - val_loss: 0.7888
Epoch 5/15
383/383 ————— 10s 17ms/step - accuracy: 0.9153 - loss: 0.2312 - val_accuracy: 0.8518 - val_loss: 0.4437
Epoch 6/15
383/383 ————— 7s 17ms/step - accuracy: 0.9254 - loss: 0.2063 - val_accuracy: 0.8580 - val_loss: 0.4223
Epoch 7/15
383/383 ————— 7s 18ms/step - accuracy: 0.9301 - loss: 0.1944 - val_accuracy: 0.8820 - val_loss: 0.3316
Epoch 8/15
383/383 ————— 7s 17ms/step - accuracy: 0.9324 - loss: 0.1881 - val_accuracy: 0.8770 - val_loss: 0.3528
Epoch 9/15
383/383 ————— 7s 17ms/step - accuracy: 0.9369 - loss: 0.1769 - val_accuracy: 0.8768 - val_loss: 0.3747
Epoch 10/15
383/383 ————— 7s 17ms/step - accuracy: 0.9456 - loss: 0.1544 - val_accuracy: 0.8589 - val_loss: 0.4220
Epoch 11/15
383/383 ————— 10s 18ms/step - accuracy: 0.9440 - loss: 0.1534 - val_accuracy: 0.9232 - val_loss: 0.2251
Epoch 12/15
383/383 ————— 7s 18ms/step - accuracy: 0.9459 - loss: 0.1469 - val_accuracy: 0.8822 - val_loss: 0.3753
Epoch 13/15
383/383 ————— 7s 17ms/step - accuracy: 0.9502 - loss: 0.1363 - val_accuracy: 0.9092 - val_loss: 0.2699
Epoch 14/15
383/383 ————— 7s 18ms/step - accuracy: 0.9518 - loss: 0.1255 - val_accuracy: 0.8987 - val_loss: 0.3209
Epoch 15/15
383/383 ————— 10s 18ms/step - accuracy: 0.9565 - loss: 0.1178 - val_accuracy: 0.8736 - val_loss: 0.4302
ResNet time: 155.31702828407288

```

```

print("\nTraining Inception...")
t0 = time.time()
histories['inception'] = inception_model.fit(X_train, y_train,
                                             epochs=EPOCHS,
                                             batch_size=BATCH_SIZE,
                                             validation_data=(X_val,
y_val),
                                             verbose=1)

times['inception'] = time.time() - t0
print("Inception time:", times['inception'])

```

```

Training Inception...
Epoch 1/15
383/383 ----- 21s 33ms/step - accuracy: 0.5683 - loss: 1.1560 - val_accuracy: 0.8077 - val_loss: 0.5329
Epoch 2/15
383/383 ----- 7s 11ms/step - accuracy: 0.8125 - loss: 0.5118 - val_accuracy: 0.8509 - val_loss: 0.4268
Epoch 3/15
383/383 ----- 5s 11ms/step - accuracy: 0.8547 - loss: 0.4036 - val_accuracy: 0.8530 - val_loss: 0.4020
Epoch 4/15
383/383 ----- 5s 11ms/step - accuracy: 0.8691 - loss: 0.3584 - val_accuracy: 0.8861 - val_loss: 0.3193
Epoch 5/15
383/383 ----- 5s 11ms/step - accuracy: 0.8827 - loss: 0.3190 - val_accuracy: 0.8907 - val_loss: 0.3000
Epoch 6/15
383/383 ----- 5s 11ms/step - accuracy: 0.8984 - loss: 0.2785 - val_accuracy: 0.8895 - val_loss: 0.3036
Epoch 7/15
383/383 ----- 4s 11ms/step - accuracy: 0.8993 - loss: 0.2703 - val_accuracy: 0.9070 - val_loss: 0.2621
Epoch 8/15
383/383 ----- 4s 11ms/step - accuracy: 0.9070 - loss: 0.2552 - val_accuracy: 0.9091 - val_loss: 0.2513
Epoch 9/15
383/383 ----- 5s 11ms/step - accuracy: 0.9113 - loss: 0.2441 - val_accuracy: 0.9030 - val_loss: 0.2661
Epoch 10/15
383/383 ----- 4s 11ms/step - accuracy: 0.9159 - loss: 0.2271 - val_accuracy: 0.9020 - val_loss: 0.2645
Epoch 11/15
383/383 ----- 5s 12ms/step - accuracy: 0.9198 - loss: 0.2182 - val_accuracy: 0.9202 - val_loss: 0.2325
Epoch 12/15
383/383 ----- 4s 11ms/step - accuracy: 0.9239 - loss: 0.2064 - val_accuracy: 0.9070 - val_loss: 0.2530
Epoch 13/15
383/383 ----- 5s 11ms/step - accuracy: 0.9203 - loss: 0.2132 - val_accuracy: 0.9145 - val_loss: 0.2447
Epoch 14/15
383/383 ----- 5s 11ms/step - accuracy: 0.9284 - loss: 0.1930 - val_accuracy: 0.9170 - val_loss: 0.2326
Epoch 15/15
383/383 ----- 4s 11ms/step - accuracy: 0.9333 - loss: 0.1854 - val_accuracy: 0.9180 - val_loss: 0.2302
Inception time: 90.77783417701721

```

```

print("\nRecorded training times keys:", list(times.keys()))

print("\nEvaluating models on test set:")
for name, model in [('alexnet', alexnet_model),
                    ('resnet', resnet_model),
                    ('inception', inception_model)]:
    loss, acc = model.evaluate(X_test, y_test, verbose=0)
    tt = times.get(name, None) # safe lookup
    if tt is None:
        tt_str = "N/A" # training time not recorded
    else:
        tt_str = f"{tt:.1f}s"
    print(f"{name:10s} loss={loss:.4f} acc={acc:.4f}")
train_time={tt_str}")

```

Recorded training times keys: ['resnet', 'inception', 'alexnet']

Evaluating models on test set:

alexnet	loss=0.2766	acc=0.9240	train_time=564.5s
resnet	loss=0.4320	acc=0.8724	train_time=155.3s
inception	loss=0.2290	acc=0.9170	train_time=90.8s